JARKOMLAN • TUGAS 2 PRAKTIKUM

Panduan Demonstrasi Kolaboratif (Host 1 & Host 2)

Skenario Duet, Daftar Perintah Eksekusi, dan Naskah Tanya-Jawab Pengujian Sistem IoT Terdistribusi

---

1. Pembagian Peran & Topologi Sistem

Komponen	Host 1 (Laptop 1)	Host 2 (Laptop 2)
Peran Utama	Pusat Backend, Broker, Database, & Dashboard di VM	Edge IoT Publisher Node & Client Monitor
Lingkungan Operasi	Docker WSL 2 & VirtualBox Linux VM (Alpine)	Docker Desktop / Local Python
Service Berjalan	• Mosquitto MQTT Broker (1883) • MySQL 8.0 DBMS (3306) • MQTT Subscriber Worker • Flask Web Server Dashboard (8080)	• MQTT Publisher Container (`jarkom-mqtt-publisher-host2`) • Web Browser Chrome / Firefox
Port Terbuka	1883 (TCP), 3306 (TCP), 8080 (TCP)	Tidak ada port masuk (Outbound Client only)

2. Persiapan Konektivitas Jaringan

Pilih salah satu metode koneksi di bawah ini sesuai situasi demo:

Opsi A: Demo Offline (Tatap Muka & Hotspot HP Bersama)

1. Salah satu orang menyalakan **Hotspot HP**. Kedua laptop wajib connect ke Wi-Fi yang sama.
2. Di **Host 1**, ketik `ipconfig` di PowerShell untuk melihat IP Wi-Fi (misal: `192.168.43.15`).
3. Di **Host 2**, buka `tugas-2/host2/docker/docker-compose.yml` dan atur: `MQTT_BROKER_HOST: "192.168.43.15"` dan `MQTT_BROKER_PORT: "1883"`.
4. Dashboard dibuka Host 2 di browser via `http://192.168.43.15:8080`.

Opsi B: Demo Online (Jarak Jauh / Port Tunneling Internet)

1. Di **Host 1**, jalankan `tugas-2\start_internet_tunnel.bat` (double-click di Windows).
2. Akan terbuka 2 jendela CMD. Salin alamat tunnel publik yang muncul:
   - Tunnel TCP (Port 1883): misal `tcp://gcdyg-182-8-97-149.run.pinggy-free.link:36219`
   - Tunnel HTTPS (Port 8080): misal `https://kqaem-2404-c0-b601.run.pinggy-free.link`
3. Di **Host 2**, masukkan Host & Port TCP tersebut ke `docker-compose.yml`:
   `MQTT_BROKER_HOST: "gcdyg-182-8-97-149.run.pinggy-free.link"`
   `MQTT_BROKER_PORT: "36219"`.
4. Host 2 dan Penguji membuka Web Dashboard langsung via link HTTPS tunnel di atas.

3. Skenario Demonstrasi Duet (Alur Per Babak)

Babak 1: Pendahuluan & Penjelasan Arsitektur (~1 Menit)

> • **HOST 1 BERBICARA:**
>
> *"Selamat pagi/siang Bapak/Ibu Dosen dan Mas/Mbak Asisten. Pada Tugas 2 JARKOMLAN ini, kami mengimplementasikan arsitektur pipeline IoT terdistribusi multi-host. Host 1 bertindak sebagai sentral backend yang mengelola MQTT Broker Mosquitto, Database MySQL, Subscriber Worker pada Docker WSL 2, serta Web Server Dashboard yang kami isolasi di dalam Virtual Machine Linux Alpine."*

> • **HOST 2 BERBICARA:**
>
> *"Sedangkan Host 2 bertindak sebagai Edge IoT Node yang menjalankan container Docker Publisher untuk mengirimkan telemetri sensor lingkungan seperti suhu, kelembapan, dan tegangan secara periodik ke Host 1 menggunakan protokol MQTT."*

Babak 2: Pembuktian Jaringan & Isolasi Web Server di VM Linux (~2 Menit)

Aksi di Layar Host 1

1. Buka terminal VirtualBox VM Alpine dan periksa antarmuka serta gateway:

```
# Tunjukkan IP eth0 VM:
ip addr show eth0

# Tunjukkan Default Gateway Host Windows:
ip route show
```

> • **HOST 1 BERBICARA:**
>
> *"Dapat dilihat pada interface eth0, VM Linux kami menerima IP privat 10.0.2.15/24 dari DHCP VirtualBox mode NAT. Melalui perintah ip route show, default gateway mengarah ke 10.0.2.2, yang merupakan representasi host fisik Windows tempat database berada."*

2. Uji Konektivitas Layer 3 (ICMP) vs Layer 4 (TCP Netcat):

```
# Uji Layer 3 (ICMP Ping):
ping -c 2 10.0.2.2

# Uji Layer 4 (Port Service Check):
nc -zv 10.0.2.2 3306
nc -zv 10.0.2.2 1883
```

> • **HOST 1 BERBICARA:**
>
> *"Ping membuktikan konektivitas kabel/Layer 3 ke host dengan RTT stabil di bawah 1 ms. Selanjutnya dengan nc -zv, kami memverifikasi keterbukaan port TCP Layer 4. Port 3306 untuk MySQL dan 1883 untuk Broker Mosquitto berstatus open dan siap melayani koneksi soket."*

Babak 3: Pembuktian Publisher Docker & Kontrak JSON (~1.5 Menit)

Aksi di Layar Host 2

Tunjukkan konfigurasi `docker-compose.yml` dan skrip `publisher.py` di Laptop 2:

> • **HOST 2 BERBICARA:**
>
> *"Pada Host 2, service publisher kami dikemas dalam Dockerfile berbasis python:3.11-slim. Kami mengarahkan target broker ke alamat Host 1 dengan topik jarkom/sensor/telemetry dan interval transmisi 3 detik. Sebelum integrasi, kami telah menetapkan kontrak normalisasi data formal sehingga format JSON yang dikirimkan terstandarisasi dan langsung diterima dengan aman oleh skema database di Host 1."*

Babak 4: Demonstrasi Live Integrasi Real-Time (~2.5 Menit)

Langkah 1: Host 1 dan Host 2 sama-sama membuka Web Dashboard di browser masing-masing.

Langkah 2: Host 2 menyalakan Publisher di terminal Laptop 2:

```
docker compose up -d
docker logs -f jarkom-mqtt-publisher-host2
```

Langkah 3: Host 1 menunjukkan log live subscriber di PowerShell Laptop 1:

```
docker logs -f jarkom-mqtt-subscriber
```

> • **HOST 1 & HOST 2 BERBICARA SERENTAK:**
>
> *"Dapat dilihat pada layar terminal Host 1, subscriber langsung menerima payload dari HOST2-IOT-NODE dan menyimpannya ke MySQL. Dan secara bersamaan pada dashboard browser di kedua laptop, angka total data serta baris tabel bertambah secara realtime setiap 3 detik tanpa perlu merefresh halaman."*

4. Pertanyaan Penguji & Jawaban Teknis Tepat

Q1: Kenapa Web Server diletakkan di VM Linux, bukan langsung di Docker Host 1?

Jawaban: Sesuai instruksi penugasan, penempatan Web Server di VM menguji kemampuan administrasi jaringan Linux murni (Alpine), pengelolaan antarmuka kernel eth0, routing gateway NAT 10.0.2.2, serta isolasi keamanan antar lingkungan server fisik dan virtual.

Q2: Bagaimana Web Server di VM (10.0.2.15) bisa mengakses MySQL (3306) di Windows Host?

Jawaban: VirtualBox mode NAT secara arsitektur menetapkan IP 10.0.2.2 sebagai alias host fisik Windows. Di Windows Host, port 3306 Docker di-bind ke 0.0.0.0:3306 dan firewall telah kami izinkan, sehingga query TCP dari dalam VM diteruskan langsung ke container MySQL.

Q3: Kenapa Host 2 tidak langsung konek dan insert ke MySQL Host 1 saja?

Jawaban: Dalam best practice keamanan dan arsitektur IoT modern, database tidak boleh diekspos langsung ke perangkat luar. Menggunakan Message Broker MQTT memberikan loose coupling, efisiensi bandwidth tinggi pada jaringan edge, serta ketahanan buffer data (fault tolerance).

Q4: Mengapa tadi saat port tunneling harus menggunakan 127.0.0.1 bukan localhost?

Jawaban: Di Windows 10/11, OpenSSH me-resolve localhost ke alamat IPv6 loopback (::1). Karena port forwarding VirtualBox dan Docker hanya listen pada IPv4 (0.0.0.0), forwarding ke IPv6 ditolak (connection refused). Penggunaan 127.0.0.1 memaksa rute IPv4 murni sehingga koneksi tunnel berhasil 100%.